

LECTURE 8

Join and Division Operations

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

Agenda

- Theta join (θ -join) — Cartesian product plus an arbitrary comparison condition
- Equijoin — the special case of theta join using only equality
- Natural join ($\bowtie$) — an equijoin with the duplicate attribute removed
- Outer joins — left, right, and full
- Division ($\div$) — answering “find every X related to all of a given set of Y”

The Extended Schema

PropertyForRent (propertyNo, street, city, type, rooms, rent, ownerNo, staffNo, branchNo) — note PG55, staffNo NULL

propertyNo	city	type	rooms	rent	ownerNo	staffNo	branchNo
PA14	Lahore	House	6	22000	CO46	SG14	B003
PL94	Islamabad	Flat	4	15000	CO87	SA9	B007
PG4	Lahore	House	5	35000	CO40	SG14	B003
PG36	Lahore	Flat	3	24500	CO93	SG37	B003
PG16	Islamabad	House	6	45000	CO93	SA9	B007
PG21	Karachi	Flat	4	28000	CO87	SL21	B005
PG55	Lahore	Flat	3	19000	CO46	(NULL)	B003

Viewing (clientNo, propertyNo, viewDate, comment) — composite primary key

clientNo	propertyNo	viewDate	comment
CR56	PA14	2026-05-24	too small
CR56	PG4	2026-06-01	too remote
CR56	PG36	2026-06-15	(NULL)
CR62	PA14	2026-05-26	(NULL)
CR62	PG4	2026-06-14	no dining room
CR76	PG4	2026-06-20	too expensive

Theta Join (θ -Join)

$R \bowtie_{\theta} S \equiv \sigma_{\theta} (R \times S)$ — θ can be $=, \neq, <, >, \leq, \geq$

Query: which (client, Lahore property) pairs can the client actually afford — $\text{maxRent} \geq \text{rent}$?

```
Client ⋈ Client.maxRent ≥ PropertyForRent.rent (σ city='Lahore' (PropertyForRent))
```

Client

clientNo	maxRent
CR56	25000
CR62	30000
CR76	20000

Lahore properties

propertyNo	rent
PA14	22000
PG4	35000
PG36	24500

Output — 4 of 9 pairs survive

clientNo	propertyNo
CR56	PA14
CR56	PG36
CR62	PA14
CR62	PG36

Tracing: CR56(25000) vs PA14(22000)✓, PG4(35000)✗, PG36(24500)✓. CR62(30000) vs PA14✓, PG4✗, PG36✓. CR76(20000) vs PA14✗, PG4✗, PG36✗.
CR76 disappears entirely — none of the three Lahore properties fit their 20,000 budget.

Equijoin

A theta join where θ is restricted to equality — the most common join, used whenever matching a foreign key to the primary key it references.

`Staff ⋈ Staff.branchNo = Branch.branchNo Branch`

Staff

staffNo	name	branchNo
SL21	John White	B005
SG37	Ann Beech	B003
SG14	David Ford	B003
SA9	Mary Howe	B007
SL41	Julie Lee	B005

Branch

branchNo	city
B003	Lahore
B005	Karachi
B007	Islamabad

Output — branchNo appears TWICE

staffNo	name	branchNo	branchNo	city
SL21	White	B005	B005	Karachi
SG37	Beech	B003	B003	Lahore
SG14	Ford	B003	B003	Lahore
SA9	Howe	B007	B007	Islamabad
SL41	Lee	B005	B005	Karachi

The defining flaw: the redundant column branchNo is listed twice, and by construction every value in both columns is identical. Carrying it around is wasteful. Natural join exists to fix this.

Natural Join (⋈)

An equijoin on ALL attributes the two relations have in common (by name), with the duplicate column dropped. Plain ⋈ with no condition almost always means natural join.

Staff ⋈ Branch

Output — one branchNo column, same 5 rows

staffNo	name	branchNo	city
SL21	John White	B005	Karachi
SG37	Ann Beech	B003	Lahore
SG14	David Ford	B003	Lahore
SA9	Mary Howe	B007	Islamabad
SL41	Julie Lee	B005	Karachi

Theta join

R×S, filtered by any comparison operator



Equijoin

Theta join, $\theta = "="$ — keeps BOTH matched columns



Natural join

Equijoin on shared names — duplicate column removed

Outer Joins: the Problem with Inner Join

Every join so far is an inner join: an unmatched tuple simply vanishes. Outer joins keep unmatched rows and pad the missing side with NULL.

Staff ⌘ PropertyForRent

Inner join — both SL41 (no properties) and PG55 (no staff) vanish

staffNo	name	propertyNo	city	rent
SG14	David Ford	PA14	Lahore	22000
SG14	David Ford	PG4	Lahore	35000
SA9	Mary Howe	PL94	Islamabad	15000
SA9	Mary Howe	PG16	Islamabad	45000
SG37	Ann Beech	PG36	Lahore	24500
SL21	John White	PG21	Karachi	28000

Julie Lee (SL41) manages no properties at all; PG55 has no assigned staff (staffNo = NULL). Each outer join variant restores exactly one gap — or both.

LEFT OUTER JOIN (⋈)

Keeps every tuple of the left relation, padding with NULL where no match exists on the right.

Staff ⋈ PropertyForRent

Every staff member kept, even with zero properties

staffNo	name	propertyNo	city	rent
SG14	David Ford	PA14	Lahore	22000
SG14	David Ford	PG4	Lahore	35000
SA9	Mary Howe	PL94	Islamabad	15000
SA9	Mary Howe	PG16	Islamabad	45000
SG37	Ann Beech	PG36	Lahore	24500
SL21	John White	PG21	Karachi	28000
SL41	Julie Lee	(NULL)	(NULL)	(NULL)

SL41 is back, NULL filling every column that would have come from PropertyForRent — nothing on the left is ever dropped.

RIGHT OUTER JOIN (⋈)

Symmetric to left outer join: keeps every tuple of the RIGHT relation instead.

Staff ⋈ **PropertyForRent**

Every property kept, even with no assigned staff

staffNo	name	propertyNo	city	rent
SG14	David Ford	PA14	Lahore	22000
SG14	David Ford	PG4	Lahore	35000
SA9	Mary Howe	PL94	Islamabad	15000
SA9	Mary Howe	PG16	Islamabad	45000
SG37	Ann Beech	PG36	Lahore	24500
SL21	John White	PG21	Karachi	28000
(NULL)	(NULL)	PG55	Lahore	19000

PG55 (rent 19,000, no assigned staff) is back; SL41 is gone again — right outer join only ever protects rows from the right relation.

FULL OUTER JOIN

Keeps every tuple from BOTH relations, padding with NULL on whichever side has no match.

Staff ⋈ **PropertyForRent**

Nothing from either side is ever dropped — 8 rows total

staffNo	name	propertyNo	city	rent
SG14	David Ford	PA14	Lahore	22000
SG14	David Ford	PG4	Lahore	35000
SA9	Mary Howe	PL94	Islamabad	15000
SA9	Mary Howe	PG16	Islamabad	45000
SG37	Ann Beech	PG36	Lahore	24500
SL21	John White	PG21	Karachi	28000
SL41	Julie Lee	(NULL)	(NULL)	(NULL)
(NULL)	(NULL)	PG55	Lahore	19000

Picking fast: lose no left rows → left outer. Lose no right rows → right outer. Complete audit of both → full outer.

Division (÷)

Answers: “find every value of A associated, in every single case, with ALL of the values in some set of B.” Given R(A,B) and S(B), $R \div S$ returns every a such that (a,b) exists in R for every tuple b in S.

Query: which clients have viewed EVERY property located in Lahore?

Divisor: Lahore propertyNo

propertyNo
PA14
PG4
PG36

Dividend: π clientNo, propertyNo (Viewing)

clientNo	propertyNo
CR56	PA14
CR56	PG4
CR56	PG36
CR62	PA14
CR62	PG4
CR76	PG4

Output: clientNo

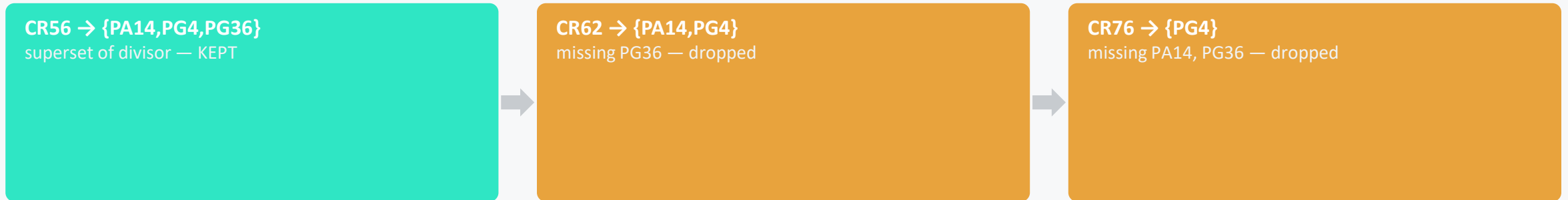
clientNo
CR56

π clientNo, propertyNo (Viewing) $\div$ π propertyNo (σ city='Lahore' (PropertyForRent))

Only CR56 has viewed all three Lahore properties. Division has no notion of “mostly” — it's all-or-nothing.

Division, Worked by Hand

Group the dividend by clientNo and check whether that client's viewed-property set is a superset of the divisor {PA14, PG4, PG36}.



Division has no single native SQL keyword

Unlike SELECT, PROJECT, and the joins, division has to be built — typically with a double negation (“no Lahore property exists that this client has NOT viewed”) using nested NOT EXISTS subqueries, or GROUP BY plus HAVING COUNT(...) = (divisor count).

A solid hand-traced mental model of division from examples like this is what makes that SQL pattern eventually click.

KEY TAKEAWAYS

Join and Division Operations

- Theta join = Cartesian product + a SELECT with any comparison operator θ — the most general join.
- Equijoin restricts θ to = and always leaves a redundant duplicate column behind.
- Natural join ($\bowtie$) is an equijoin on all commonly-named attributes, with the duplicate column automatically removed — the join you'll write most often.
- Outer joins preserve unmatched rows an inner join would drop: left keeps every left row, right keeps every right row, full keeps both, all padded with NULL.
- Division ($\div$) answers “find every A related to ALL of a given set of B” — group the dividend by A and keep groups whose B-values are a superset of the divisor; no single SQL keyword, usually NOT EXISTS or GROUP BY / HAVING COUNT.